本番環境ガイド

今のファイル構成（Next.js ＋ Rails API ＋ PostgreSQL、ファイルは Amazon S3）を公開するための、デプロイ先の候補・比較・構築手順。料金と制限は 2026年10月時点の目安で、契約前に各社の公式ページで確かめること。

この資料の内容

1. 提出物のチェック表
2. 今の構成と、本番で変えるところ
3. 本番用の設定（S3・DB・環境変数）
4. 候補A：手軽さ重視（Vercel ＋ Render）
5. 候補B：AWS で統一（ECS ＋ RDS）
6. 比べる表とおすすめ
7. 公開後の確認と運用
8. 参考資料と、料金の確認先

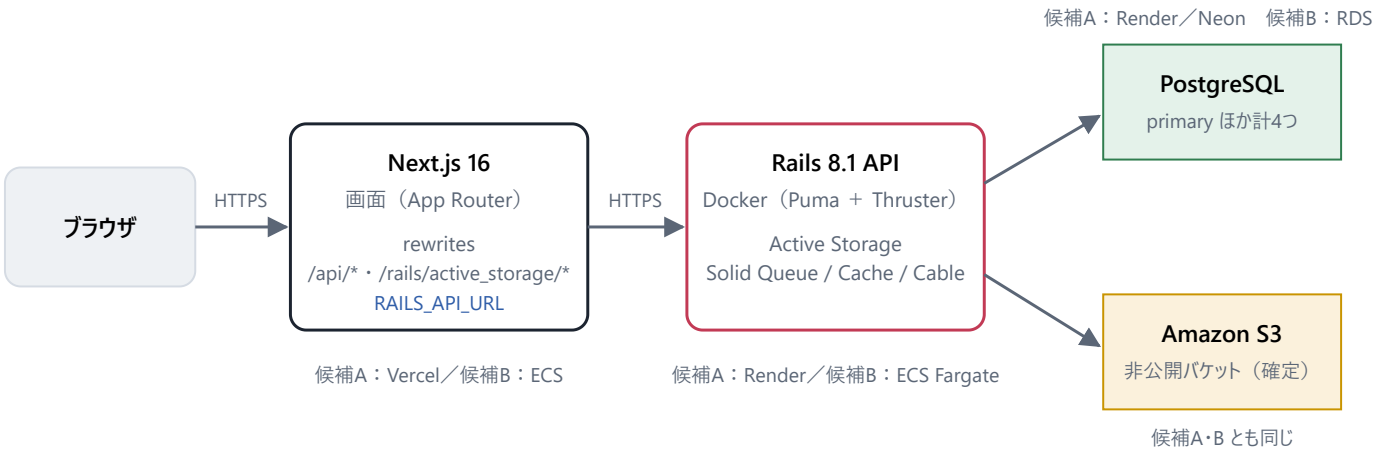
1. 提出物のチェック表

項目	区分	状態	置き場所・内容
README.md	必須	作成	`README.md`：概要・主な機能・技術構成・ディレクトリ構成・起動方法・環境変数・テスト・資料の一覧
.gitignore	必須	整備	ルートの `.gitignore`：`.env*`・`*.key`・`*.pem`・`node_modules`・`.next`・ログ・OS のファイル。backend・frontend にもそれぞれの .gitignore。`master.key` が追跡されていないことを確認済み
コミット履歴	必須	整理	未コミットだった変更を、API・テスト・共通処理・UI 部品・プレビュー・ビューア・画面・資料・CI の単位に分けてコミット
ER図	推奨	作成	`docs/er-diagram.md`（Mermaid。GitHub 上で図として表示）、`docs/database-design.md`（列・制約・削除時の扱い）
システム構成図	推奨	更新	`docs/architecture.md`：開発と本番の構成図、一覧・登録・ファイル表示の流れ、セキュリティ方針。この資料の2章にも本番の図
API仕様書	推奨	更新	`docs/api-spec.md`：19本の API の URL・メソッド・リクエスト・レスポンス・エラー
テスト手順	推奨	作成	`docs/testing.md`：自動テストのコマンドと内容、画面での確認項目（登録・フォルダ・まとめ・マップ・検索・ビューア・お気に入り）
GitHub Actions	発展	拡張	`.github/workflows/ci.yml`：Rails（Brakeman・bundler-audit・RuboCop・テスト）に、Next.js（ESLint・型チェック・ビルド）を追加。Dependabot に npm を追加

> 本番環境の構築（この資料の3〜5章）はまだ行っていない。どちらの候補でも、まず3章の「本番用の設定」をコードに入れてから始める。

2 今の構成と、本番で変えるところ

ブラウザが話す相手は Next.js だけ。Next.js が API とファイルの要求を Rails へ転送する。



今のファイル構成から読み取れること

ファイル	今の状態	本番での意味
`frontend/next.config.ts`	rewrites で `/api/*` と `/rails/active_storage/*` を `RAILS_API_URL` へ転送。転送する本文の上限を 520MB に広げている	Next.js のホスティング先で `RAILS_API_URL` を**ビルド時**に渡す。CORS 設定は不要
`backend/Dockerfile`	Rails 標準。Ruby 4.0.7、Thruster が 80 番で待ち受け、Puma（3000 番）へ渡す。起動時に `db:prepare`	コンテナを動かせるサービスならそのまま使える。マイグレーションは起動時に自動で走る
`backend/config/database.yml`	production は primary・cache・queue・cable の4つの DB	DB を4つ作るか、接続先の環境変数を4つ渡す（3章）
`backend/config/storage.yml`	amazon の設定がコメントアウト	S3 を有効にする（3章）
`config/environments/production.rb`	保存先 `:local`。SSL の強制はコメントアウト。ジョブは Solid Queue	保存先を `:amazon` に、SSL とホストの制限を有効に
`backend/config/deploy.yml`	Kamal の雛形のまま	候補B の別案（Lightsail ＋ Kamal）で使う
API の `file_url`	`rails_storage_proxy_path`（proxy 方式）	ファイルは S3 → Rails → Next.js → ブラウザの順に流れる。S3 を公開しなくてよい

アップロードの上限に注意。アップロードは Next.js の rewrites を通って Rails に届く。Vercel のサーバーレス関数の本文上限は 4.5MB で、外部への rewrites にも上限がかかるかどうかは公式資料で明記されていない。候補A では公開前に 10MB・100MB のファイルで必ず試し、通らなければ「S3 への直接アップロード」（7章）に切り替える。

3 本番用の設定（どちらの候補でも共通）

先にコードと AWS 側を整えておく。秘密情報はリポジトリに入れない。

3.1 S3 のバケットと IAM

1. S3 でバケットを作る（例 `materialorg-production`、リージョン **ap-northeast-1（東京）**）。「パブリックアクセスをすべてブロック」はオンのまま。暗号化（SSE-S3）とバージョニングを有効にする。
2. IAM で Rails 用のユーザー（候補B ではタスクロール）を作り、このバケットだけを読み書きできるポリシーを付ける。

```
{ "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::materialorg-production" },
    { "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::materialorg-production/*" } ] }
```

3.2 Rails の設定

```
# backend/Gemfile
gem "aws-sdk-s3", require: false

# backend/config/storage.yml（amazon を有効にする）
amazon:
  service: S3
  access_key_id: <%= ENV["AWS_ACCESS_KEY_ID"] %>          # 候補B のタスクロールでは2行とも削除
  secret_access_key: <%= ENV["AWS_SECRET_ACCESS_KEY"] %>
  region: ap-northeast-1
  bucket: <%= ENV.fetch("S3_BUCKET", "materialorg-production") %>

# backend/config/environments/production.rb
config.active_storage.service = :amazon
config.assume_ssl = true      # TLS はホスティング側で終端される
config.force_ssl = true
config.ssl_options = { redirect: { exclude: ->(request) { request.path == "/up" } } }
config.hosts = [ ENV.fetch("APP_HOST") ]                      # 例 api.example.com
config.host_authorization = { exclude: ->(request) { request.path == "/up" } }
```

3.3 データベース（4つ）

Rails 8 の production は、業務データ（primary）に加えて Solid Cache・Solid Queue・Solid Cable の DB を使う。マネージド PostgreSQL を1台用意し、その中に DB を4つ作るのが安い。

```
CREATE DATABASE materialorg_production;
CREATE DATABASE materialorg_production_cache;
CREATE DATABASE materialorg_production_queue;
CREATE DATABASE materialorg_production_cable;
```

接続先は環境変数で渡す（Rails は `DATABASE_URL` を primary に、`CACHE_DATABASE_URL` などを同名の接続に重ねる）。添付ファイルの解析・削除はジョブで動くので、`SOLID_QUEUE_IN_PUMA=true` で Puma の中でジョブも処理する。

3 本番用の設定（続き）

ホスティング先の画面（またはシークレット管理）に登録する値。

3.4 環境変数

変数	設定先	値の例・説明
`RAILS_MASTER_KEY`	Rails	`backend/config/master.key` の中身。シークレットとして登録
`DATABASE_URL`	Rails	`postgres://user:pass@host:5432/materialorg_production?sslmode=require`
`CACHE_DATABASE_URL` / `QUEUE_DATABASE_URL` / `CABLE_DATABASE_URL`	Rails	同じホストの `..._cache`・`..._queue`・`..._cable`
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `S3_BUCKET`	Rails	3.1 の IAM ユーザーの鍵（候補B ではタスクロールを使い不要）
`APP_HOST`	Rails	Rails の公開ホスト名（`config.hosts` 用）
`SOLID_QUEUE_IN_PUMA`	Rails	`true`
`RAILS_LOG_LEVEL`	Rails	`info`（調査時は `debug`）
`RAILS_API_URL`	Next.js	Rails の URL（`https://api.example.com`）。**ビルド時**に必要

3.5 コードに入れる変更のまとめ

ファイル	変更
`backend/Gemfile`	`aws-sdk-s3` を追加して `bundle install`（Gemfile.lock もコミット）
`backend/config/storage.yml`	`amazon` を有効にし、リージョンを東京に
`backend/config/environments/production.rb`	保存先 `:amazon`、`assume_ssl`・`force_ssl`、`config.hosts`
`frontend/Dockerfile`（候補B のみ）	`next.config.ts` に `output: 'standalone'` を足し、`node server.js` で起動するイメージを作る
`.github/workflows/`（任意）	main への push でデプロイするジョブ（候補A は各サービスの GitHub 連携で不要）

3.6 秘密情報の扱い

- `master.key`・AWS の鍵・DB のパスワードはリポジトリに入れない（ルートと backend の .gitignore で除外済み）。
- 鍵は各サービスの環境変数（シークレット）か、AWS Secrets Manager に入れる。
- AWS の鍵は 3.1 のバケット専用の権限だけにする。漏れたときはすぐに無効化して作り直す。
- ルートユーザーは使わず、MFA を有効にした IAM ユーザーで作業する。

4 候補A：手軽さ重視（Vercel ＋ Render ＋ S3）

画面の操作が中心で、サーバーの管理がほぼ要らない。GitHub への push で自動デプロイされる。

役割	サービス	選ぶ理由
Next.js	Vercel（Hobby → Pro）	Next.js の開発元。rewrites・キャッシュ・プレビュー環境がそのまま動く
Rails API	Render の Web Service（Docker）	既存の `backend/Dockerfile` をそのまま使える。HTTPS と独自ドメインが自動
PostgreSQL	Render Postgres（または Neon）	Rails と同じリージョンに置ける。Neon は無料枠で試しやすい
ファイル	Amazon S3（東京）	確定。3章の設定を使う

手順

1. **3章の設定をコミットして GitHub へ push する**

   aws-sdk-s3・storage.yml・production.rb の変更。S3 バケットと IAM ユーザーも作っておく。

2. **Render で PostgreSQL を作る**

   New → Postgres。リージョンは Singapore（日本に最も近い）。作成後、外部接続の URL で `psql` に入り、3.3 の `CREATE DATABASE` を4つ実行する。Neon を使う場合も同じく4つ作り、接続文字列に `sslmode=require` を付ける。

3. **Render で Rails を作る**

   New → Web Service → GitHub のリポジトリを選ぶ。

   - Language：**Docker**、Root Directory：`backend`、Region：DB と同じ
   - Docker Command：`./bin/rails server -b 0.0.0.0`（Render が渡す `$PORT` で Puma が直接待ち受ける。Thruster は使わない）
   - Health Check Path：`/up`
   - 環境変数：3.4 の Rails の分をすべて登録（DB は Render の Internal URL）

   デプロイすると、起動時に `bin/docker-entrypoint` が `db:prepare` を実行し、4つの DB にテーブルを作る。

4. **Vercel で Next.js を作る**

   Add New → Project → 同じリポジトリ。Root Directory：`frontend`（Framework は自動で Next.js）。Environment Variables に `RAILS_API_URL=https://（Render の URL）` を Production・Preview の両方に入れてからデプロイする。

5. **独自ドメイン（任意）**

   画面を `app.example.com`（Vercel）、API を `api.example.com`（Render）に割り当て、`APP_HOST` と `RAILS_API_URL` を更新して再デプロイする。

6. **アップロードの確認**

   10MB・100MB・500MB のファイルを登録してみる。Vercel で 413 などが出るときは、7章の「S3 への直接アップロード」に切り替える。

費用の目安（月額）

構成	Vercel	Render（Rails）	DB	S3	合計
試作（個人・非商用）	Hobby 無料	Free（15分で休止）	Neon 無料（0.5GB）	数十円〜	**ほぼ 0 円**
常時公開	Hobby 無料	Starter $7	Render Basic-256mb $6	10GB で数十円	**約 $13〜**

Render の無料の Web Service は使われないと休止し、次の表示に時間がかかる。無料の Render Postgres は30日で使えなくなる。Vercel Hobby は個人・非商用の利用に限られる。

5 候補B：AWS で統一（ECS Fargate ＋ RDS ＋ S3）

S3 と同じ AWS の中で完結し、ネットワークと権限を細かく管理できる。手間と費用は上がる。



> ALB がパスで振り分けるので、アップロードは Next.js を通らずに Rails へ直接届く（2章のアップロードの上限の心配がない）。Next.js の rewrites は開発環境用としてそのまま残してよい。

手順

1. **ネットワーク**
   VPC を作る（パブリックサブネット2つ・プライベートサブネット2つ、ap-northeast-1a/1c）。費用を抑えるなら、ECS タスクをパブリックサブネットに置き NAT ゲートウェイを使わない（セキュリティグループで ALB からの通信だけを許可）。

2. **RDS for PostgreSQL**
   db.t4g.micro・シングル AZ・ストレージ 20GB から。プライベートサブネットに置き、ECS のセキュリティグループからの 5432 番だけを許可。作成後に 3.3 の DB を4つ作る。自動バックアップは7日。

3. **イメージを ECR へ**
   リポジトリ `materialorg-backend`・`materialorg-frontend` を作る。Rails は `backend/Dockerfile`、Next.js は `output: 'standalone'` を使う `frontend/Dockerfile` を新たに追加してビルドし、push する。

4. **シークレットと権限**
   `RAILS_MASTER_KEY` と DB の URL を Secrets Manager に入れる。Rails のタスクロールに 3.1 の S3 ポリシーを付ける（鍵の環境変数は不要）。

5. **ECS（Fargate）**
   クラスターを作り、タスク定義を2つ作る（Rails：0.5 vCPU / 1GB・ポート 80、Next.js：0.25 vCPU / 0.5GB・ポート 3000）。サービスを作り、ALB のターゲットグループにつなぐ。ヘルスチェックは Rails が `/up`。

6. **ALB とドメイン**
   ACM で証明書を取り、HTTPS リスナーを作る。ルール：`/api/*`・`/rails/*`・`/up` → Rails、それ以外 → Next.js。Route 53 でドメインを ALB へ向ける。

7. **自動デプロイ**
   GitHub Actions に OIDC で AWS のロールを渡し、main への push でイメージをビルド → ECR へ push → ECS のサービスを更新する。

安く小さく始める別案：Lightsail ＋ Kamal

Lightsail のインスタンス（2GB）1台に、既存の `backend/config/deploy.yml`（Kamal）で Rails を置く。kamal-proxy が HTTPS 証明書を自動で取る。DB は Lightsail のマネージド DB か同じ台の PostgreSQL、画面は Vercel。月 $12〜$27 程度で、サーバーの更新は自分で行う。

6 比べる表とおすすめ

◎ とてもよい　○ よい　△ 弱い。費用は小さな構成での月額の目安。

観点	候補A Vercel ＋ Render	候補B ECS ＋ RDS	別案 Lightsail ＋ Kamal
月額の目安	0〜約 $13	約 $50〜80 ALB・Fargate 2つ・RDS	約 $12〜27
始めるまでの手間	◎	△	○
運用の手間（OS 更新など）	◎	○	△
大きなファイルのアップロード	△ 要確認・直接アップロードで解決	◎	◎
S3 との相性（権限・通信）	○ IAM ユーザーの鍵	◎ タスクロール・同じ AWS 内	○
利用者が増えたときの伸ばしやすさ	○	◎	△
就職活動などでの説明のしやすさ	○	◎ VPC・IAM・ECS の経験	○
日本からの近さ	○ Render は Singapore	◎ 東京	◎ 東京

おすすめ：まず候補A で公開し、必要になったら候補B へ移る

1. 今は利用者が1人で、ログイン機能もない。候補A なら費用をほぼかけずに、数時間で公開できる。
2. アプリはすでに Docker（Rails）と環境変数で動く形なので、候補B へ移るときもコードの変更はほとんどない。S3 は最初から本番と同じものを使う。
3. 公開前に必ずアップロードの上限を試す。大きな動画を扱うなら、7章の直接アップロードを先に入れるか、最初から候補B を選ぶ。
4. AWS の経験を形にしたい場合は、候補B を「学習用の本番」として並行して作るのも有効（費用の上限アラートを必ず設定）。

移行するときの流れ（A → B）

1. RDS を作り、Render の DB を `pg_dump` → `pg_restore` で移す（4つとも。cache・cable は空から作り直してもよい）。
2. S3 はそのまま使う（バケットも鍵も変えない。B ではタスクロールに切り替える）。
3. ECS で動かして確認できたら、DNS を ALB へ切り替え、Render と Vercel を止める。

7 公開後の確認と運用

公開した直後に必ず通す確認と、続けて行うこと。

7.1 公開直後の確認

#	確認すること	方法と期待する結果
1	Rails の起動	`curl -i https://api.example.com/up` が 200
2	画面から API へ届く	画面を開き、フォルダー一覧が出る（ブラウザの開発者ツールで `/api/v1/folders` が 200）
3	S3 への保存	PDF を登録し、S3 のバケットにオブジェクトが増える。`active_storage_blobs.service_name` が `amazon`
4	ファイルの配信	ビューアで PDF・画像が開く。S3 の URL を直接開いても見られない（非公開）
5	Range（位置合わせ）	動画・音声のシークで、その位置から再生される（応答が 206）
6	大きなファイル	100MB・500MB の登録が通る。通らなければ 7.3 を行う
7	削除	教材を削除すると、少し後に S3 のオブジェクトも消える（Solid Queue のジョブ）
8	HTTPS	`http://` で開くと `https://` へ移る

7.2 運用

- **バックアップ**：DB の自動バックアップ（Render の有料 DB・RDS は標準）。S3 はバージョニングで誤削除に備える。
- **費用**：AWS Budgets で月の上限（例 $20）を超えそうなときにメールを送る。Render・Vercel も使用量の通知を有効に。
- **ログ**：Render のログ／CloudWatch Logs。`RAILS_LOG_LEVEL=info`。
- **CI との連携**：GitHub Actions の CI（RuboCop・Brakeman・テスト・ESLint・型チェック・ビルド）が通ったものだけを main にマージし、main への push で自動デプロイ。Vercel はプルリクエストごとにプレビューの URL を作る。
- **依存の更新**：Dependabot（bundler・npm・Actions）のプルリクエストを毎週確認する。
- **公開範囲**：今はログインがない。URL を知っていれば誰でも操作できるため、公開するなら Vercel のパスワード保護・Basic 認証・ログイン機能のどれかを先に入れる。

7.3 大きなファイル向け：S3 への直接アップロード

ブラウザが Rails から署名付きの URL を受け取り、ファイル本体を S3 へ直接送る方式。Next.js と Rails がファイル本体を受け取らないので、上限とメモリの心配がなくなる。

1. フロントで Active Storage の `DirectUpload`（`@rails/activestorage`）を使い、`/rails/active_storage/direct_uploads` から URL を受け取ってアップロードする。
2. 教材の登録は、ファイルの代わりに `signed_id` を `material[file]` に入れて送る（API の形はほぼそのまま）。
3. S3 のバケットに CORS を設定する。

```
[ { "AllowedOrigins": ["https://app.example.com"],
    "AllowedMethods": ["PUT"],
    "AllowedHeaders": ["Content-Type", "Content-MD5", "Content-Disposition"],
    "MaxAgeSeconds": 3600 } ]
```

配信も S3 から直接行う（redirect 方式）場合は、`GET` と `Range` ヘッダーも許可し、API の `file_url` を `rails_blob_path` に変える。

8 参考資料と、料金の確認先

料金・無料枠・制限は変わりやすい。下記の公式ページで最新の値を確かめてから契約する。

内容	URL	この資料で使った値
Render の料金	https://render.com/pricing	Web Service Starter $7、Postgres Basic-256mb $6、無料 Postgres は30日
Render の Docker デプロイ	https://render.com/docs/docker	Root Directory・Docker Command
Neon の料金・無料枠	https://neon.com/pricing	無料：0.5GB・100 CU 時間／月
Vercel の rewrites	https://vercel.com/docs/rewrites	外部へのリライト
Vercel Functions の上限	https://vercel.com/docs/functions/limitations	本文 4.5MB
Vercel の料金	https://vercel.com/pricing	Hobby は個人・非商用
Amazon S3 の料金	https://aws.amazon.com/s3/pricing/	東京の保存・転送料
AWS Fargate の料金	https://aws.amazon.com/fargate/pricing/	vCPU・メモリ時間
Amazon RDS for PostgreSQL	https://aws.amazon.com/rds/postgresql/pricing/	db.t4g.micro
AWS Pricing Calculator	https://calculator.aws/	候補B の見積もり
Amazon Lightsail の料金	https://aws.amazon.com/lightsail/pricing/	インスタンス・マネージド DB
Rails：Active Storage	https://guides.rubyonrails.org/active_storage_overview.html	S3 の設定・直接アップロード・proxy 方式
Rails：複数データベース	https://guides.rubyonrails.org/active_record_multiple_databases.html	`*_DATABASE_URL`
Kamal	https://kamal-deploy.org/	Lightsail の別案
Next.js の standalone 出力	https://nextjs.org/docs/app/api-reference/config/next-config-js/output	候補B の frontend/Dockerfile

リポジトリの関連資料

`README.md`	概要・起動方法・環境変数
`docs/architecture.md`	システム構成図とデータの流れ
`docs/er-diagram.md`	ER図
`docs/api-spec.md`	API仕様書
`docs/testing.md`	テスト手順（本番の確認は 7.1）
`.github/workflows/ci.yml`	CI（Rails・Next.js）

この資料の費用は 2026年10月時点で調べた目安で、為替・税・データ転送量で変わる。候補B は AWS Pricing Calculator で見積もってから作る。